
WakeKV: Reactive, Reversible KV Residency for Heads That Change Their Minds

Utkarsh Ranjan
UC San Diego
utranjan@ucsd.edu

Abstract

1 Most KV-cache compression methods classify attention heads once, either offline
2 or during prefill, and keep this classification fixed throughout generation. Across
3 three models (1.5B–8B) and three regimes (needle retrieval, long chain-of-thought,
4 and multi-turn recall), we measure head behavior on four model-regime combina-
5 tions and find that most heads change their reading behavior at least once during
6 generation. We introduce *WakeKV*, a reactive residency policy that moves cooling
7 heads to a recoverable CPU reservoir rather than freezing or permanently evict-
8 ing their state. At matched memory or budget, *WakeKV* consistently improves
9 miss rate over frozen classification and destructive eviction—evaluated across five
10 model-regime combinations—and over three cited baselines (SnapKV, uniform
11 R-KV, and ReasonAlloc) across four eligible combinations. A FlexiCache/vLLM
12 implementation on Mistral-7B confirms the benefit on real hardware, improving
13 throughput while retaining LongBench quality.

14 1 Introduction

15 KV-cache size dominates long-context LLM serving memory: caching every attended key/value
16 pair for every head grows linearly in sequence length and quickly exceeds GPU memory at scale,
17 so a serving system must decide which parts of the cache to keep resident, compress, or evict. The
18 dominant response decides each head’s treatment once, offline or at prefill, and holds that decision
19 fixed for the rest of generation.

20 Existing approaches split along two axes: *when* a head’s role is decided, and *whether* a demoted head’s
21 state can be recovered. Static head-role methods (DuoAttention [7], RazorAttention [8], MoA [9],
22 HeadKV [10], FlexiCache [2], HeteroCache [3]) and adaptive allocators (Ada-KV [11], CAKE [12],
23 LAVa [13], DynamicKV [14]) both fix treatment offline or at prefill; the adaptive allocators do so
24 despite being framed as dynamic. Decode-time compressors (R-KV [6], G-KV [15], ThinKV [16],
25 SCOPE [17]) update during generation but evict destructively; ReasonAlloc [4] reallocates per-head
26 budgets online but still destroys KV on demotion. Offload systems (ArkVale [18], ShadowKV [19],
27 InfiniGen [20], HeadInfer [21], FlexiCache, HeteroCache) keep demoted KV recoverable but never
28 revisit head priority at runtime. Among the systems we survey, none is both *dynamic* and *reversible*
29 at once (Figure 1a).

30 Recent measurement work on retrieval heads [1] casts doubt on this shared premise: attention-head
31 importance is non-stationary even within a single generation, and that work names dynamic per-head
32 KV retention as an open direction. We close that gap with *WakeKV* (§3), a reactive residency
33 policy that never permanently discards a head’s state, and validate it across five real model/regime
34 combinations at matched memory and budget (§5).

35 Our contributions include:

- 36 1. **Measurement:** We show that attention-head behavior changes during generation, across three
37 models and three regimes, including two that competing frozen-role methods do not evaluate.
- 38 2. **Prediction study:** We find that inexpensive online signals do not reliably anticipate these changes.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

39 3. **WakeKV:** We introduce reactive, reversible KV residency and show consistent gains over frozen
 40 classification, destructive eviction, and three cited baselines, at matched memory or budget.
 41 4. **System evaluation:** On A30/Mistral-7B via a FlexiCache/vLLM shim, WakeKV reactive beats
 42 stock throughput at every rerank interval (1.34–2.33 $\times$) while holding 95–99% of LongBench
 43 quality, confirming the simulator-level direction on real hardware.

44 2 Motivation

45 Existing KV-cache systems often assume that an attention head’s role remains stable during generation.
 46 We ask whether heads instead change behavior over time, and whether those changes can be predicted
 47 well enough to prefetch their KV state; §4 describes the experimental setup.

48 2.1 Head Churn

49 We first measure how much the set of positions attended to by each head changes from one decoding
 50 step to the next. Let A_t denote the set of heads (or top- k pages) attended to at decode step t ; we
 51 measure step-to-step stability using the adjacent-step Jaccard index:

$$J(A_t, A_{t+1}) = \frac{|A_t \cap A_{t+1}|}{|A_t \cup A_{t+1}|}. \quad (1)$$

52 $J = 1$ means the attended set is unchanged between consecutive steps; $J = 0$ means complete
 53 turnover. Measured adjacent-step Jaccard is 0.53 (continuous score, NIAH), within [1]’s reported
 54 0.28–0.51 range (full per-regime breakdown in Table 4, Appendix).

55 We then identify “unstable” heads using FlexiCache’s Random-Corrected Overlap statistic, which
 56 accounts for the amount of overlap that could occur by chance when selecting from a pool of size N :

$$\text{RCO}(A, B) = \max\left(0, \frac{|A \cap B|/K - K/N}{1 - K/N}\right), \quad K = \min(|A|, |B|). \quad (2)$$

57 $\text{RCO} = 0$ corresponds to chance-level overlap, while $\text{RCO} = 1$ means that the two sets are identical.
 58 A head is flagged unstable when its mean RCO over a trailing 16-step window falls below 0.5.
 59 The fraction of heads with ≥ 1 HeteroCache-style drift event (windowed-median overlap dropping
 60 below $\tau=0.5$, baseline reset after each event) ranges 0.63–0.85 across regimes and holds at 8B
 61 (0.79 on CoT). These two numbers describe different things: only 2–9% of heads are *persistently*
 62 unstable (mean RCO below threshold throughout), while 63–85% experience *at least one* discrete
 63 drift event—a small chronically-unstable tail alongside a large majority of occasionally-shifting
 64 heads.

65 2.2 Wake-up Prediction

66 We grade four candidate signals (page-overlap change, windowed-median drift, entropy trend, needle-
 67 mass delta) under a *causal* per-head z-score, $z_t = (x_t - \mu_{t-w:t})/\sigma_{t-w:t}$, computed online from only
 68 the trailing $w=32$ -step window’s mean and standard deviation. The best signal achieves precision
 69 0.08–0.20 at recall 0.41–0.70 (an ensemble never beats the single best). A structurally different
 70 candidate—whether the just-generated token predicts a wake burst (Bonferroni-corrected, held out)—
 71 reaches F1 0.53–0.62 on CoT, 4–5 $\times$ every attention-derived signal, but zero markers survive on
 72 NIAH/multi-turn. Wake-ups are also bursty ($z = +9.7$ vs. shuffle null), so a coarse trigger only
 73 catches 54% of them at a 2%-of-steps rate. Neither cheap proactive route proved reliable enough to
 74 prefetch on in our experiments.

75 Together, these findings motivate WakeKV (§3): heads shift too much for a one-time classification,
 76 but not predictably enough to prefetch—so WakeKV reacts to demand instead of anticipating it.

77 3 Method

78 WakeKV replaces per-head classification with a single runtime rule: each head keeps a GPU-resident
 79 set, LRU-ordered and capped at a per-head budget B ; an evicted page is demoted to a CPU reservoir
 80 rather than destroyed, and is fetched back the moment it is wanted again (Figure 1b). A wrong or
 81 stale classification thus costs one recoverable stall, not a permanent accuracy loss.

82 Concretely, each head maintains a GPU-resident set ordered by recency, a CPU reservoir holding
 83 every demoted page’s full, uncompressed key/value state, and a budget B capping GPU residency.

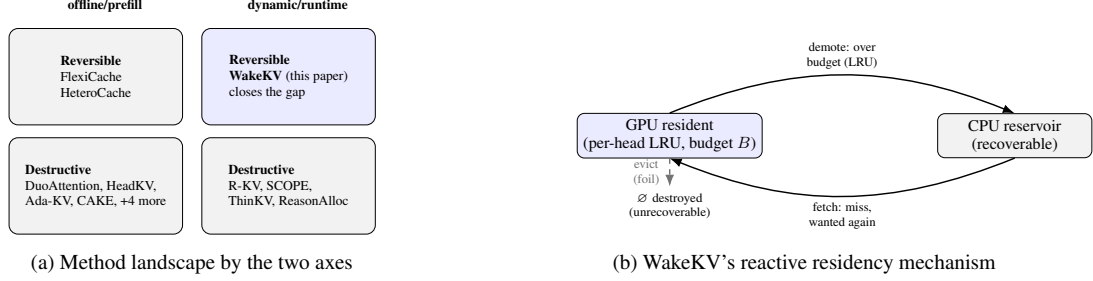


Figure 1: (a) Prior methods split along two axes—when role is decided, and whether demotion is reversible—leaving the dynamic-and-reversible quadrant empty until WakeKV. (b) A demoted head’s state moves to a recoverable CPU reservoir instead of being destroyed; the *evict* foil (dashed) shares WakeKV’s exact LRU schedule but destroys state instead of reserving it.

Each step’s attention over resident pages defines *demand*: read pages move to the front of the LRU order. When demand pushes the resident set past B , the least-recently-used page is *demoted*—copied to the reservoir, not discarded. A later want of a non-resident page is a *miss*: its exact state is fetched back before that step’s attention completes, at the cost of one PCIe transfer, with nothing about its history lost. This single rule both narrows a head’s working set (via B) and restores it on demand (via fetch) without ever committing to a permanent classification.

4 Evaluation Setup

We evaluate on three models, Qwen2.5-3B-Instruct, DeepSeek-R1-Distill-Qwen-1.5B, and DeepSeek-R1-Distill-Llama-8B, across three regimes: NIAH (needle retrieval, ≈ 5000 -token filler), long CoT (MATH-500 [22], up to 2048 generated tokens), and multi-turn recall. At every decode step and head we log the top- k attended positions and weights ($k = 64$), along with a continuous needle score and a binary copy-paste score following [1]’s Eq. 1–2. The 1.5B and 3B models were measured on an RTX 2080 Ti (11GB); the 8B model and the real-system follow-up ran on an A30 (24GB). These logs support the analyses in §2 and §5 (which adds a fifth combo, Mistral-7B/NIAH); the real-hardware deployment runs live on a separate A30.

5 Results

Having established that heads shift substantially and cheap prediction cannot reliably anticipate wake events (§2), we ask whether reacting to demand pays off: we simulate WakeKV against a frozen classifier and a destructive-eviction foil sharing its exact LRU schedule, then extend to three baselines and a real-hardware deployment. Because two policies at the same nominal budget can realize different memory, every comparison below holds memory or budget fixed rather than a shared label. Table 1 summarizes the headline result; details follow.

Comparison (matched memory or budget)	Coverage	WakeKV wins
vs. Frozen, equal memory	5 model/regime	17/17
vs. Evict, equal budget	5 model/regime	19/20
vs. SnapKV/R-KV/ReasonAlloc, equal memory	4 model/regime [†]	32/32

Table 1: Headline results: how often reactive’s miss rate was $\leq$ the comparator’s, at every point where memory or budget was matched exactly rather than compared at a shared nominal label (full breakdown in Table 3; real-system result in Table 2). [†]Mistral-7B/NIAH excluded: its R-KV/ReasonAlloc needed a non-default pruning interval.

For example, at $\sim 56k$ matched pages on Mistral/NIAH, reactive’s miss rate is 0.055 against frozen’s 0.405 ($7\times$ lower); the one loss against evict is 8B/CoT at $B=8$ (0.632 vs. 0.624); and at $\sim 28k$ matched pages on 8B/CoT, reactive’s 0.078 beats R-KV’s 0.132.

5.1 Reactive vs. Frozen and Evict

We simulate four policies at page granularity over the same logged attention: *full* (ceiling), *frozen* (FlexiCache’s published defaults), *reactive* (WakeKV, §3), and *evict* (reactive’s exact LRU schedule, but destructive demotion, isolating reversibility). Sweeping budget B traces each policy’s (memory,

miss-rate) curve (Figure 2); since frozen and reactive reach different memory at the same B , we interpolate reactive’s curve onto frozen’s 17 measured memory values (pooled across combos, dropping 3 of 20 points that reactive never reached). At those 17 matched-memory levels, reactive’s miss rate is $\leq$ frozen’s: **17/17**. At matched budget (no interpolation needed), reactive beats evict in **19/20** settings; the sole exception, at the tightest 8B/CoT budget, is where recovering a page that evict would have destroyed alters the subsequent resident set.

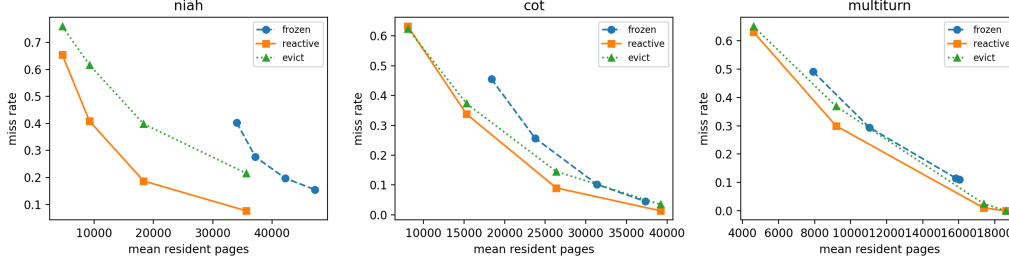


Figure 2: Memory-vs-miss-rate Pareto fronts across NIAH, CoT, and multi-turn. Reactive sits at or below frozen’s at all 17 matched-memory points and below evict’s at 19 of 20 matched budgets (Table 1).

We further compare against simulator-compatible reimplementations of three cited methods—SnapKV [5], uniform R-KV [6], and ReasonAlloc [4] (each a declared simplification; details in Appendix A)—at matched rather than nominal memory, since all three prune periodically and can use several times reactive’s memory at the same B (Table 3, “Reactive mem”). Reactive wins **32/32** comparisons across the four eligible combos—e.g. on 8B/CoT at ~ 28 k pages of realized memory, R-KV’s miss rate is 0.132 against reactive’s 0.078 (Table 1[†]).

5.2 Real-Hardware Deployment

A FlexiCache/vLLM shim turns stock FlexiCache into WakeKV reactive: zero heads classified “stable,” reranking every step instead of its native cadence. The strict reactive configuration (rerank every step, $R=1$) is WakeKV’s design point, beating stock throughput by $1.34\times$ at 97.4% LongBench quality. Relaxing how often WakeKV rechecks head residency trades strictness for throughput, tracing a frontier that reaches $2.33\times$ at $R=16$ while quality stays above 95% (Table 2). Part of the win is less attention work, not purely memory: zero dense heads at stock’s cadence alone reach that same $2.33\times$.

Config	Rerank int.	Gen tok/s	vs stock	LongBench %
Stock FlexiCache	16 (native)	114.4	1.00 $\times$	100
Reactive (WakeKV)	1	153.6	1.34$\times$	97.4
Reactive	2	193.3	1.69$\times$	97.1
Reactive	4	226.3	1.98$\times$	99.1
Reactive	8	254.2	2.22$\times$	97.9
Reactive	16*	266.6	2.33$\times$	95.0

Table 2: Real-system throughput $\times$ quality sweep (A30, Mistral-7B, LEval 8k $\rightarrow$ 1000; LongBench, 4×30 samples). **Bold** marks speedup over stock; $R=1$ is WakeKV’s design point, * $R=16$ its Pareto knee (quality stays above 95%).

6 Conclusion and Future Work

Head importance need not be classified once and frozen. A single reactive rule—demote under pressure, fetch on demand, never discard—consistently beats both the frozen and destructive alternatives (Table 1), and the gain carries from simulation through to real hardware.

Several limitations remain: our evaluation is limited to 1.5B–8B models with uneven regime coverage; the simulator counts *misses*, not stall *time*, against FlexiCache-style policies rather than exact code; and the real-system result is narrow (one model/regime, one A30). A natural next step is comparing reversible and destructive eviction on a real system using stall time. Still, reactive, reversible residency is a promising alternative to fixed per-head classification.

References

- [1] Yuping Lin, Zitao Li, Yue Xing, Pengfei He, Yingqian Cui, Yaliang Li, Bolin Ding, Jingren Zhou, Jiliang Tang. Retrieval Heads are Dynamic. arXiv:2602.11162 (ACL 2026).
- [2] Nazmul Takbir, Hamidreza Alikhani, Nikil Dutt, Sangeetha Abdu Jyothi. FlexiCache: Leveraging Temporal Stability of Attention Heads for Efficient KV Cache Management. arXiv:2511.00868v2 (MLSys 2026).
- [3] Zhiyuan Shi, Qibo Qiu, Feng Xue, Zhonglin Jiang, Li Yu, Jian Jiang, Xiaofei He, Wenxiao Wang. HeteroCache: A Dynamic Retrieval Approach to Heterogeneous KV Cache Compression for Long-Context LLM Inference. arXiv:2601.13684v2 (ACL 2026 Main).
- [4] Wenhao Liu, Hao Shi, Yunhe Li, Weizhi Fei, Xiangyuan Wang, Mengzhe Ruan, Hanxu Hou, Peisong Wang, Linqi Song, Shuang Qiu. ReasonAlloc: Hierarchical Decoding-Time KV Cache Budget Allocation for Reasoning Models. arXiv:2606.11164v1.
- [5] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, Deming Chen. SnapKV: LLM Knows What You are Looking for Before Generation. arXiv:2404.14469 (NeurIPS 2024).
- [6] Zefan Cai, Wen Xiao, Hanshi Sun, Cheng Luo, Yikai Zhang, Ke Wan, Yucheng Li, Yeyang Zhou, Li-Wen Chang, Jiuxiang Gu, Zhen Dong, Anima Anandkumar, Abedelkadir Asi, Junjie Hu. R-KV: Redundancy-aware KV Cache Compression for Reasoning Models. arXiv:2505.24133.
- [7] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, Song Han. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. arXiv:2410.10819 (ICLR 2025).
- [8] Hanlin Tang, Yang Lin, Jing Lin, Qingsen Han, Shikuan Hong, Yiwu Yao, Gongyi Wang. RazorAttention: Efficient KV Cache Compression Through Retrieval Heads. arXiv:2407.15891.
- [9] Tianyu Fu, Haofeng Huang, Xuefei Ning, Genghan Zhang, Boju Chen, Tianqi Wu, Hongyi Wang, Zixiao Huang, Shiyao Li, Shengen Yan, Guohao Dai, Huazhong Yang, Yu Wang. Mixture of Attention Spans: Optimizing LLM Inference Efficiency with Heterogeneous Sliding-Window Lengths. arXiv:2406.14909 (CoLM 2025).
- [10] Yu Fu, Zefan Cai, Abedelkadir Asi, Wayne Xiong, Yue Dong, Wen Xiao. Not All Heads Matter: A Head-Level KV Cache Compression Method with Integrated Retrieval and Reasoning. arXiv:2410.19258 (ICLR 2025).
- [11] Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, S. Kevin Zhou. Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. arXiv:2407.11550 (NeurIPS 2025).
- [12] Ziran Qin, Yuchen Cao, Mingbao Lin, Wen Hu, Shixuan Fan, Ke Cheng, Weiyao Lin, Jianguo Li. CAKE: Cascading and Adaptive KV Cache Eviction with Layer Preferences. arXiv:2503.12491 (ICLR 2025).
- [13] Yiqun Shen, Song Yuan, Zhengze Zhang, Xiaoliang Wang, Daxin Jiang, Nguyen Cam-Tu. LAVa: Layer-wise KV Cache Eviction with Dynamic Budget Allocation. arXiv:2509.09754.
- [14] Xiabin Zhou, Wenbin Wang, Minyan Zeng, Jiaxian Guo, Xuebo Liu, Li Shen, Min Zhang, Liang Ding. DynamicKV: Task-Aware Adaptive KV Cache Compression for Long Context LLMs. arXiv:2412.14838 (EMNLP 2025 Findings).
- [15] Mengqi Liao, Lu Wang, Chaoyun Zhang, Zekai Shen, Xiaowei Mao, Si Qin, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, Huaiyu Wan. G-KV: Decoding-Time KV Cache Eviction with Global Attention. arXiv:2512.00504.
- [16] Akshat Ramachandran, Marina Neseem, Charbel Sakr, Rangharajan Venkatesan, Brucek Khailany, Tushar Krishna. ThinKV: Thought-Adaptive KV Cache Compression for Efficient Reasoning Models. arXiv:2510.01290 (ICLR 2026, Oral).
- [17] Jialong Wu, Zhenglin Wang, Linhai Zhang, Yilong Lai, Yulan He, Deyu Zhou. SCOPE: Optimizing Key-Value Cache Compression in Long-context Generation. arXiv:2412.13649 (ACL 2025).
- [18] Renze Chen, Zhuofeng Wang, BeiQuan Cao, Tong Wu, Size Zheng, Xiuhong Li, Xuechao Wei, Shengen Yan, Meng Li, Yun Liang. ArkVale: Efficient Generative LLM Inference with Recallable Key-Value Eviction. NeurIPS 2024.

- 195 [19] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong,
196 Yuejie Chi, Beidi Chen. ShadowKV: KV Cache in Shadows for High-Throughput Long-Context
197 LLM Inference. arXiv:2410.21465 (ICML 2025, Spotlight).
- 198 [20] Wonbeom Lee, Jungi Lee, Junghwan Seo, Jaewoong Sim. InfiniGen: Efficient Generative
199 Inference of Large Language Models with Dynamic KV Cache Management. arXiv:2406.19707
200 (OSDI 2024).
- 201 [21] Cheng Luo, Zefan Cai, Hanshi Sun, Jinqi Xiao, Bo Yuan, Wen Xiao, Junjie Hu, Jiawei Zhao,
202 Beidi Chen, Anima Anandkumar. HeadInfer: Memory-Efficient LLM Inference by Head-wise
203 Offloading. arXiv:2502.12574.
- 204 [22] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan
205 Leike, John Schulman, Ilya Sutskever, Karl Cobbe. Let’s Verify Step by Step. arXiv:2305.20050.
206 (Introduces the MATH-500 test subset used in this paper.)

207 Appendix

208 A Full Per-Combo, Per-Budget Sweep

209 Our three baselines are simulator-compatible reimplementations, each a declared simplification of
 210 its cited method: SnapKV [5] (prefill-only top-token selection, frozen through decode), uniform
 211 R-KV [6] (periodic redundancy-aware pruning, reduced to importance-only since raw keys aren’t
 212 logged), and ReasonAlloc [4] (online per-head reallocation, calibrated with an equal split rather than
 213 its offline Reasoning-Wave allocation).

214 Table 3 gives the raw miss rate for every policy at every swept *nominal* budget $B \in \{8, 16, 32, 64\}$,
 215 across all five model/regime combos; this is the source Table 1’s win-counts are computed from. Only
 216 Reactive and Evict are directly comparable within a row, since they share realized memory (identical
 217 LRU cap). Frozen/SnapKV/R-KV/ReasonAlloc realize very different memory at the same nominal
 218 B , so Table 1’s matched-memory comparison—not this table—is the appropriate read for those four
 219 columns. Mistral-7B/NIAH’s R-KV/ReasonAlloc columns (marked †) additionally use a non-default
 220 pruning interval (buffer/delta = 8, not the default 128) to avoid a degenerate no-pruning result on
 221 this model’s short NIAH answers, and are excluded from Table 1’s win-counts.

Model/regime	B	Frozen	Reactive	Evict	SnapKV	R-KV	ReasonAlloc	Reactive mem
Qwen-3B/multiturn	8	0.491	0.631	0.651	0.428	0.163	0.163	4,608
	16	0.294	0.300	0.369	0.247	0.093	0.090	9,187
	32	0.116	0.010	0.025	0.201	0.010	0.009	17,373
	64	0.110	0.000	0.000	0.201	0.000	0.000	18,621
Qwen-3B/NIAH	8	0.403	0.655	0.760	0.499	0.250	0.250	4,608
	16	0.276	0.409	0.617	0.429	0.179	0.177	9,209
	32	0.197	0.187	0.398	0.374	0.128	0.126	18,337
	64	0.155	0.076	0.215	0.368	0.080	0.074	35,638
R1-8B/CoT	8	0.455	0.632	0.624	0.021	0.508	0.514	8,088
	16	0.257	0.338	0.373	0.007	0.314	0.317	15,298
	32	0.102	0.090	0.145	0.005	0.132	0.129	26,362
	64	0.045	0.014	0.035	0.005	0.031	0.027	39,178
R1-1.5B/CoT	8	0.395	0.542	0.543	0.032	0.399	0.401	2,637
	16	0.191	0.221	0.263	0.014	0.201	0.199	4,899
	32	0.066	0.040	0.074	0.012	0.063	0.058	8,014
	64	0.042	0.005	0.013	0.012	0.012	0.009	10,692
Mistral-7B/NIAH	8	0.579	0.650	0.761	0.689	0.421 [†]	0.422 [†]	8,192
	16	0.506	0.443	0.613	0.559	0.296 [†]	0.303 [†]	16,349
	32	0.442	0.180	0.308	0.442	0.167 [†]	0.170 [†]	32,168
	64	0.407	0.048	0.090	0.421	0.065 [†]	0.047 [†]	57,514

Table 3: Raw simulator sweep: miss rate per policy at each nominal budget B (page units), plus reactive’s realized memory (“Reactive mem”). Reactive vs. Evict, the one same-row-comparable pair, favors reactive in 19 of 20 rows (the exception is 8B/CoT, $B=8$); see §5 for why the other four columns require Table 1’s matched-memory correction instead. [†]Non-default pruning interval (see text above); excluded from Table 1’s win-counts.

222 B Head Churn Metrics by Regime

223 Table 4 gives the full per-regime breakdown behind §2’s head-churn numbers: adjacent-step Jaccard,
 224 the fraction of heads that are *persistently* unstable (mean RCO below threshold throughout), and the
 225 fraction that experience *at least one* HeteroCache-style drift event.

Regime	Model	Adj. Jaccard	Persistently unstable	Ever drifted (≥ 1 event)
NIAH	Qwen2.5-3B	0.53 (cont.)	0.064	0.63
CoT	R1-Distill-1.5B	0.82 (bin.)	0.023	0.85
Multi-turn	Qwen2.5-3B	0.79 (bin.)	0.085	0.76
CoT	R1-Distill-8B	0.93 (bin.)	0.042	0.79

Table 4: Head-shift metrics by regime. Adjacent Jaccard: continuous score (NIAH), binary score (CoT/multi-turn). At 8B (bottom row), binary Jaccard tightens as copy behavior stabilizes on math CoT, but the page-level drift metric (what a residency controller reacts to) stays high at 0.79, so the case is undiminished at scale.